

ML307C

外围接口开发指导手册

版本：V1.0.0

发布日期：2025/3/6

服务与支持

如果您有任何关于模组产品及产品手册的评论、疑问、想法，或者任何无法从本手册中找到答案的疑问，请通过以下方式联系我们。

OneMO官网： onemo10086.com

邮箱： SmartModule@cmiot.chinamobile.com

客户服务热线： 400-110-0866



中国移动
China Mobile

文档声明

注意

本手册描述的产品及其附件特性和功能，取决于当地网络设计或网络性能，同时也取决于用户预先安装的各种软件。由于当地网络运营商、ISP，或当地网络设置等原因，可能也会造成本手册中描述的全部或部分产品及其附件特性和功能未包含在您的购买或使用范围之内。

责任限制

除非合同另有约定，中移物联网有限公司对本文档内容不做任何明示或暗示的声明或保证，并且不对特定目的适销性及适用性或者任何间接的、特殊的或连带的损失承担任何责任。

在适用法律允许的范围内，在任何情况下，中移物联网有限公司均不对用户因使用本手册内容和本手册中描述的产品而引起的任何特殊的、间接的、附带的或后果性的损坏、利润损失、数据丢失、声誉和预期的节省而负责。

因使用本手册中所述的产品而引起的中移物联网有限公司对用户的最大赔偿（除在涉及人身伤害的情况中根据适用法律规定的损害赔偿外），不应超过用户为购买此产品而支付的金额。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。公司保留随时修改本手册中任何信息的权利，无需进行提前通知且不承担任何责任。

商标声明



为中国移动注册商标。

本手册和本手册描述的产品中出现的其他商标、产品名称、服务名称和公司名称，均为其各自所有者的财产。

进出口法规

出口、转口或进口本手册中描述的产品（包括但不限于产品软件和技术数据），用户应遵守相关进出口法律和法规。

隐私保护

关于我们如何保护用户的个人信息等隐私情况，请查看相关隐私政策。

操作系统更新声明

操作系统仅支持官方升级；如用户自己刷非官方系统，导致安全风险和损失由用户负责。

固件包完整性风险声明

固件仅支持官方升级；如用户自己刷非官方固件，导致安全风险和损失由用户负责。

版权所有©中移物联网有限公司。保留一切权利。

本手册中描述的产品，可能包含中移物联网有限公司及其存在的许可人享有版权的软件，除非获得相关权利人的许可，否则，非经本公司书面同意，任何单位和个人不得擅自摘抄、复制本手册内容的部分或全部，并以任何形式传播。



中国移动
China Mobile

关于文档

修订记录

版本	描述
V1.0.0	初版



中国移动
China Mobile

目录

服务与支持.....	ii
文档声明.....	iii
关于文档.....	v
1. 概述.....	8
1.1. 适用范围.....	8
1.2. 写作目的.....	8
2. 外设接口总述.....	9
3. GPIO.....	10
3.1. 硬件说明.....	10
3.2. API说明.....	11
3.3. 使用示例.....	16
3.4. 注意事项.....	17
4. ADC.....	18
4.1. 硬件说明.....	18
4.2. API说明.....	18
4.3. 使用示例.....	20
4.4. 注意事项.....	20
5. UART.....	21
5.1. 硬件说明.....	21
5.2. API说明.....	22
5.3. 使用示例.....	27
5.4. 注意事项.....	30
6. SPI.....	31
6.1. 硬件说明.....	31
6.2. API说明.....	32
6.3. 使用示例.....	35
6.4. 注意事项.....	41
7. I2C.....	42
7.1. 硬件说明.....	42
7.2. API说明.....	43
7.3. 使用示例.....	45
7.4. 注意事项.....	48
8. PWM.....	49
8.1. 硬件说明.....	49
8.2. API说明.....	50
8.3. 使用示例.....	52
8.4. 注意事项.....	53
9. KEYPAD.....	54
9.1. 硬件说明.....	54
9.2. API说明.....	55
9.3. 使用示例.....	58
9.4. 注意事项.....	58



中国移动
China Mobile

1. 概述

本文档介绍了OpenCPU SDK外围接口资源的硬件说明、API说明、使用示例以及注意事项，外围接口包括GPIO、I2C、UART、PWM等。

1.1. 适用范围

Table 1. 适用模组

模组系列	模组子型号
ML307C	ML307C-DC-CN

1.2. 写作目的

本文档介绍了模组OpenCPU的外围接口函数的使用示例和使用注意事项，帮助软件开发者更好理解和使用SDK的接口函数，实现外设相关功能开发。

2. 外设接口总述

本文档主要介绍了外设软件接口，通过这些接口可以实现外设的连接，但是在使用这些函数接口之前，需要阅读相关硬件手册实现硬件的正确连接，在使用时需要仔细阅读外设的注意事项，以便实现外设的正确连接和操作。



中国移动
China Mobile

3. GPIO

本章主要介绍OpenCPU SDK中外设GPIO的相关使用情况。

3.1. 硬件说明

OpenCPU GPIO引脚映射详见对应版本的资源综述和硬件设计手册。

3.2. API说明

SDK中提供一套完整的GPIO用户编程接口，具体接口定义和说明请查阅cm_gpio.h头文件，并以SDK为准，这里不再赘述。

GPIO参数结构体

```
/** 引脚ID,详情参照资源综述 */
typedef enum{
    CM_GPIO_NUM_0,
    CM_GPIO_NUM_1,
    ...
    CM_GPIO_NUM_MAX
}cm_gpio_num_e;

/** *GPIO工作模式 */
typedef enum
{
    CM_GPIO_MODE_NUM/*!<不可用 工作模式数量*/
}cm_gpio_mode_e;

/** I/O方向 */
typedef enum{
    CM_GPIO_DIRECTION_INPUT = 0,
    CM_GPIO_DIRECTION_OUTPUT,
}cm_gpio_direction_e;

/** 上/下拉 */
typedef enum{
    CM_GPIO_PULL_NONE,
    CM_GPIO_PULL_DOWN,
    CM_GPIO_PULL_UP,
}cm_gpio_pull_e;

/** 边沿+电平触发 */
typedef enum{
    CM_GPIO_IT_EDGE_RISING,
    CM_GPIO_IT_EDGE_FALLING,
    CM_GPIO_IT_EDGE_BOTH,
    CM_GPIO_IT_LEVEL_HIGH/*!<不支持*/
    CM_GPIO_IT_LEVEL_LOW/*!<不支持*/
}cm_gpio_interrupt_e;

/** 高低电平 */
typedef enum{
    CM_GPIO_LEVEL_LOW,
    CM_GPIO_LEVEL_HIGH,
}cm_gpio_level_e;

/** 命令码 */
```

```
typedef enum{
    CM_GPIO_CMD_SET_PULL, /*!< 上下拉设置命令码*/
    CM_GPIO_CMD_GET_PULL, /*!< 上下拉获取命令码*/
    CM_GPIO_CMD_SET_LEVEL, /*!< 驱动能力设置命令码*/
    CM_GPIO_CMD_GET_LEVEL, /*!< 驱动能力获取命令码*/
    CM_GPIO_CMD_SET_DIRECTION, /*!< 输入输出设置命令码*/
    CM_GPIO_CMD_GET_DIRECTION, /*!< 输入输出获取命令码*/
} cm_gpio_cmd_e;

/** 配置 */
typedef struct{
    cm_gpio_mode_e mode; /*!< 不支持*/
    cm_gpio_direction_e direction;
    cm_gpio_pull_e pull;
} cm_gpio_cfg_t;
```

GPIO接口定义

```
/**
 * @brief 初始化GPIO
 *
 * @param [in] pin GPIO引脚ID
 * @param [in] cfg 配置
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details *****初始化之前一定要先设置引脚复用*****
 */
int32_t cm_gpio_init(cm_gpio_num_e gpio_num, cm_gpio_cfg_t *cfg);

/**
 * @brief 去初始化
 *
 * @param [in] pin GPIO引脚ID
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details More details
 */
int32_t cm_gpio_deinit(cm_gpio_num_e gpio_num);

/**
 * @brief 设置输出电平
 *
 * @param [in] pin GPIO引脚ID
 * @param [in] level 输出电平
 *
 * @return
 * = 0 - 成功 \n
```

```

* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_set_level(cm_gpio_num_e gpio_num, cm_gpio_level_e level);

/**
* @brief 读取输入电平
*
* @param [in] pin GPIO引脚ID
* @param [out] level 输入电平
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_get_level(cm_gpio_num_e gpio_num, cm_gpio_level_e *level);

/**
* @brief 设置上/下拉
*
* @param [in] pin GPIO引脚ID
* @param [in] type 上下拉配置
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_set_pull(cm_gpio_num_e gpio_num, cm_gpio_pull_e type);

/**
* @brief 读取上/下拉
*
* @param [in] pin GPIO引脚ID
* @param [out] type 上下拉配置
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_get_pull(cm_gpio_num_e gpio_num, cm_gpio_pull_e *type);

/**
* @brief I/O方向模式配置
*
* @param [in] pin GPIO引脚ID

```

```

* @param [in] dir 输入/输出
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_set_direction(cm_gpio_num_e gpio_num, cm_gpio_direction_e dir);

/**
* @brief 获取I/O方向模式配置
*
* @param [in] pin GPIO引脚ID
* @param [out] dir 输入/输出
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_get_direction(cm_gpio_num_e gpio_num, cm_gpio_direction_e *dir);

/**
* @brief 注册中断回调函数
*
* @param [in] pin GPIO引脚ID
* @param [in] interrupt_cb 中断回调函数
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_interrupt_register(cm_gpio_num_e gpio_num, void *interrupt_cb);

/**
* @brief 使能中断
*
* @param [in] pin GPIO引脚ID
* @param [in] intr_mode 中断触发方式
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_gpio_interrupt_enable(cm_gpio_num_e gpio_num, cm_gpio_interrupt_e intr_mode);

```

```
/**
 * @brief 失能中断
 *
 * @param [in] pin GPIO引脚ID
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details More details
 */
int32_t cm_gpio_interrupt_disable(cm_gpio_num_e gpio_num);

/**
 * @brief GPIO控制
 *
 * @param [in] pin GPIO引脚ID
 * @param [in] cmd 命令码
 * @param [in] arg 命令
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details 根据不同的cmd值（命令码），arg参数类型须对应配置。 \n
 * cmd = \ref GPIO_CMD_SET_PULL, arg传入 \ref cm_gpio_pull_e * \n
 * cmd = \ref GPIO_CMD_GET_PULL, arg传入 \ref cm_gpio_pull_e * \n
 * cmd = \ref GPIO_CMD_SET_LEVEL, arg传入 \ref cm_gpio_level_e * \n
 * cmd = \ref GPIO_CMD_GET_LEVEL, arg传入 \ref cm_gpio_level_e * \n
 * cmd = \ref GPIO_CMD_SET_DIRECTION, arg传入 \ref cm_gpio_direction_e * \n
 * cmd = \ref GPIO_CMD_GET_DIRECTION, arg传入 \ref cm_gpio_direction_e * \n
 */
int32_t cm_gpio_ioctl(cm_gpio_num_e gpio_num, int32_t cmd, void *arg);
```

3.3. 使用示例

GPIO DEMO提供了GPIO参数设置、输入输出设置、电平设置和中断设置等功能示例程序，详细信息参考SDK中的cm_demo_gpio.c文件，并以SDK为准，这里不再赘述。

电平输入

参照SDK文件cm_demo_gpio.c中cm_test_gpio_read()函数，使用步骤：

1. 将相应PIN脚功能设置为GPIO功能，注意PIN脚号和GPIO编号的区别；
2. GPIO初始化；
3. 读取相应GPIO的电平。

电平输出

参照SDK文件cm_demo_gpio.c中cm_test_gpio_write()函数，使用步骤：

1. 将相应PIN脚功能设置为GPIO功能，注意PIN脚号和GPIO编号的区别；
2. GPIO初始化；
3. 设置相应GPIO的电平。

中断

参照SDK文件cm_demo_gpio.c中cm_test_gpio_irq()函数，使用步骤：

1. 将相应PIN脚功能设置为GPIO功能，注意PIN脚号和GPIO编号的区别；
2. GPIO初始化；
3. 注册回调函数；
4. 使能GPIO中断。

3.4. 注意事项

使用GPIO应注意以下事项：

- 大多数 引脚都是多路复用的。这些引脚也称为多功能引脚，因为它们中的每一个都分配了多个功能。每个引脚都有一个专用的多功能引脚寄存器，用于控制其功能和行为。从分配给多功能引脚的几个功能中，选择一个。其他的引脚是专用引脚，仅分配给一个功能。因此，使用前设置引脚复用情况，且同时仅有一个引脚复用为该功能，配置复用功能之前，确保每个引脚的功能都不一样；
- 中断回调函数中不能调用打印等重进入函数，不能调用耗时等任务；
- GPIO个数参照资源综述或者产品定义文档。

4. ADC

本章主要介绍OpenCPU SDK中外设ADC的相关使用情况。

4.1. 硬件说明

OpenCPU ADC引脚映射详见对应版本的资源综述和硬件设计手册。

4.2. API说明

SDK中提供一套完整的ADC用户编程接口，具体接口定义和说明请查阅cm_adc.h头文件。

ADC参数结构体

```
/** ADC设备ID */
typedef enum
{
    CM_ADC_0 = 1, /*!< ADC0 */
    CM_ADC_1, /*!< ADC1*/
} cm_adc_dev_e;
```

ADC接口定义

```
/**
 * @brief 读取ADC电压
 *
 * @param [in] dev ADC设备ID
 * @param [out] voltage 电压值, 单位: mv
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details 通过ADC设备读取测量电压值
 */
int32_t cm_adc_read(cm_adc_dev_e dev, int32_t *voltage);

/**
 * @brief 读取VBAT电压
 *
 * @param [out] voltage 电压值, 单位: mv
 *
 * @return
 * = 0 - 成功.
 * < 0 - 失败, 返回值为错误码.
 *
 * @details 通过ADC通道读取VBAT(电池)电压值
```

```
*/  
int32_t cm_adc_vbat_read(uint32_t *voltage);
```

4.3. 使用示例

ADC DEMO提供了ADC通道电压值读取功能示例程序，详细信息请参考SDK中的cm_demo_adc.c文件。

ADC通道电压读取

```
int cm_test_adc(unsigned char **cmd, int len)
{
    int32_t voltage=0;
    uint32_t voltage_vbat=0;
    int32_t ret;

    CM_DEMO_ADC_LOG("adc test start!!\n");

    ret = cm_adc_vbat_read(&voltage_vbat);
    if(ret != RET_SUCCESS)
    {
        CM_DEMO_ADC_LOG("adc vbate read err,ret=%d\n", ret);
        return -1;
    }
    CM_DEMO_ADC_LOG("adc vbate read:%ld(mv)!!\n",voltage_vbat);

    //测试ADC0
    ret = cm_adc_read(CM_ADC_0,&voltage);
    if(ret != RET_SUCCESS)
    {
        CM_DEMO_ADC_LOG("adcCM_ADC_0 read err,ret=0x%08x\n", ret);
        return -1;
    }
    CM_DEMO_ADC_LOG("adc CM_ADC_0 read:%ld(mv)!!\n",voltage);
    CM_DEMO_ADC_LOG("adc test end!!\n");

    return 0;
}
```

4.4. 注意事项

本节介绍ADC的相关注意事项。

 **Note:** ADC获取电压量程为在0~1.2V。

5. UART

本章主要介绍OpenCPU SDK中外设UART的相关使用情况。

5.1. 硬件说明

OpenCPU UART引脚映射详见对应版本的资源综述和硬件设计手册。



中国移动
China Mobile

5.2. API说明

SDK中提供一套完整的UART用户编程接口，具体接口定义和说明请查阅cm_uart.h头文件。

SDK支持两路串口。串口可配置为普通模式和低功耗模式，低功耗模式的串口在睡眠状态下可正常收发，如何配置请参考使用实例。UART0具备串口唤醒功能，如何配置请参考使用实例。SDK支持的波特率已在头文件中用宏定义列出，包括标准波特率和特殊波特率，若要使用特殊波特率请详细查看注意事项。串口采用事件驱动的形式，若注册了相应的事件，当有接收到数据时将上报接收事件，当缓存区满时将上报溢出事件。相关配置支持和说明在不同的SDK版本中可能不同，请以SDK头文件中为准。

UART可支持的配置

```

/** 标准波特率 */
#define CM_UART_BAUDRATE_300    (300U)
#define CM_UART_BAUDRATE_600    (600U)
#define CM_UART_BAUDRATE_1200   (1200U)
#define CM_UART_BAUDRATE_2400   (2400U)
#define CM_UART_BAUDRATE_4800   (4800U)
#define CM_UART_BAUDRATE_9600   (9600U)
#define CM_UART_BAUDRATE_14400  (14400U)
#define CM_UART_BAUDRATE_19200  (19200U)
#define CM_UART_BAUDRATE_38400  (38400U)
#define CM_UART_BAUDRATE_57600  (57600U)
#define CM_UART_BAUDRATE_76800  (76800U)
#define CM_UART_BAUDRATE_115200 (115200U)
#define CM_UART_BAUDRATE_230400 (230400U)
#define CM_UART_BAUDRATE_460800 (460800U)
#define CM_UART_BAUDRATE_921600 (921600U)

/** 特殊波特率 */
/* 注意事项：受分频时钟影响，当某串口配置成某一特殊波特率后，分频时钟即被修改，即之后所有配置成普通模式的串口仅支持该特殊波特率。
*
* 配置成低功耗模式的串口采用单独的分频时钟，故不受该影响，即低功耗模式的串口依然可以使用115200及以下的标准波特率（不支持76800）。
*
* 示例：若某一串口配置成256000波特率的普通模式，则之后所有配置成普通模式的串口仅支持256000波特率，而配置成低功耗模式的串口依然支持115200及以下的标准波特率（不支持76800）。
*/
#define CM_UART_BAUDRATE_256000 (256000U)
#define CM_UART_BAUDRATE_2000000 (2000000U)

/** 设备ID，详情参照资源综述*/
/* FOTA升级期间，会有数据从主串口吐出（对应ML307R为UART0），用户无需关注该数据，也不应处理该数据 */
/* 增加UART2，使用PIN38 PIN39，该引脚的默认功能是打印log，若要进行普通串口初始化，需要先用虚拟AT执行
AT+MCFG=log2cat,1以切换log打印到USB，之后再行普通串口初始化，该切换配置掉电不保存 */
typedef enum{
    CM_UART_DEV_0,    /*!< 设备0*/
    CM_UART_DEV_1,    /*!< 设备1*/
    CM_UART_DEV_2,    /*!< 设备2*/
    CM_UART_DEV_NUM
} cm_uart_dev_e;

/** 数据位 */

```

```

typedef enum{
    CM_UART_BYTE_SIZE_8 = 8,
    CM_UART_BYTE_SIZE_7 = 7,
    CM_UART_BYTE_SIZE_6 = 6,
    CM_UART_BYTE_SIZE_5 = 5,
} cm_uart_byte_size_e;

/** 奇偶校验 */
typedef enum{
    CM_UART_PARITY_NONE,
    CM_UART_PARITY_ODD,
    CM_UART_PARITY_EVEN,
    CM_UART_PARITY_MARK, /*!< 不支持*/
    CM_UART_PARITY_SPACE /*!< 不支持*/
} cm_uart_parity_e;

/** 停止位 */
typedef enum{
    CM_UART_STOP_BIT_ONE,
    CM_UART_STOP_BIT_ONE_HALF,
    CM_UART_STOP_BIT_TWO
} cm_uart_stop_bit_e;

/** 流控制 */
typedef enum{
    CM_UART_FLOW_CTRL_NONE,
    CM_UART_FLOW_CTRL_HW, /*!< 使能流控, (当前版本不支持)*/
} cm_uart_flow_ctrl_e;

/** 结果码 */
typedef enum{
    CM_UART_RET_OK = 0,
    CM_UART_RET_INVALID_PARAM = -1,
    CM_UART_RET_CTS_HIGH = -2,
} cm_uart_ret_e;

/** 事件类型 */
typedef enum
{
    CM_UART_EVENT_TYPE_RX_ARRIVED = (1 << 0), /*!< 接收到新的数据*/
    CM_UART_EVENT_TYPE_RX_OVERFLOW = (1 << 1), /*!< 接收FIFO缓存溢出*/
    CM_UART_EVENT_TYPE_TX_COMPLETE = (1 << 2), /*!< 不支持该事件*/
    CM_UART_EVENT_TYPE_RX_FLOWCTRL = (1 << 3) /*!< 流控事件(当前版本参数无效)*/
} cm_uart_event_type_e;

/** 配置 */
typedef struct{
    cm_uart_byte_size_e byte_size; /*!< 数据位, 枚举*/
    cm_uart_parity_e parity; /*!< 校验位, 枚举*/
    cm_uart_stop_bit_e stop_bit; /*!< 停止位, 枚举*/
    cm_uart_flow_ctrl_e flow_ctrl; /*!< 流控制, 枚举*/
    uint32_t baudrate; /*!< 波特率, 支持的波特率见本文件中标准波特率宏定义和特殊波特率宏定义*/
    uint32_t is_lpuart; /*!< 1:低功耗串口(仅支持115200以内的标准波特率(不支持76800)); 0:普通串口*/
    uint32_t rxrb_buf_size; /*!< 环形缓存区大小, 若为0则按默认配置8K*/
    uint32_t fc_high_threshold; /*!< (当前版本参数无效)*/
    uint32_t fc_low_threshold; /*!< (当前版本参数无效)*/
}

```

```

} cm_uart_cfg_t;

/** 事件 */
typedef struct{
    uint32_t event_type; /*!< 要注册的事件，数据可读/溢出等，注册后才会上报*/
    void *event_param; /*!< 事件参数*/
    void *event_entry; /*!< 事件执行入口*/
} cm_uart_event_t;

/* 回调函数声明 */
/* 将会上报用户参数和事件 */
typedef void (*cm_uart_event_cb_t)(void *param, uint32_t evt);

```

UART接口说明

```

/**
 * @brief 打开串口
 *
 * @param[in] dev 串口设备ID
 * @param[in] cfg 串口配置
 *
 * @return
 * = 0 - 成功\n
 * < 0 - 失败, 返回值为错误码
 *
 * @details open之前必须先对引脚复用功能进行设置。
 * 串口支持的波特率及模式见cm_uart_cfg_t结构体注释说明，请详细查看波特率宏定义处的注意事项
 */
int32_t cm_uart_open(cm_uart_dev_e dev, cm_uart_cfg_t *cfg);

/**
 * @brief 注册串口事件
 *
 * @param[in] dev 串口设备ID
 * @param[in] event 串口事件
 *
 * @return
 * = 0 - 成功\n
 * < 0 - 失败, 返回值为错误码
 *
 * @details 事件包括串口数据可读/溢出/流控等。需在open之前注册。
 * 内置8K缓存区保存未读出的数据，若注册了溢出事件，缓存区满后将上报溢出，缓存区溢出状态下将丢弃新接收的数据。
 * 回调函数中不可输出LOG、串口打印、执行复杂任务或消耗过多资源。
 */
int32_t cm_uart_register_event(cm_uart_dev_e dev, void *event);

/**
 * @brief 关闭串口
 *
 * @param[in] dev 串口设备ID
 *
 * @return
 * = 0 - 成功\n
 * < 0 - 失败, 返回值为错误码
 *
 * @details

```

```

*/
int32_t cm_uart_close(cm_uart_dev_e dev);

/**
 * @brief 串口写数据
 *
 * @param [in] dev 串口设备ID
 * @param [in] data 待写入数据
 * @param [in] len 长度
 * @param [in] timeout 超时时间(ms)(无效参数)
 *
 * @return
 * = 实际写入长度 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details
 */
int32_t cm_uart_write(cm_uart_dev_e dev, const void *data, int32_t len, int32_t timeout);

/**
 * @brief 串口读数据
 *
 * @param [in] dev 串口设备ID
 * @param [out] data 待读数据
 * @param [in] len 长度
 * @param [in] timeout 超时时间(ms)(无效参数)
 *
 * @return
 * = 实际读出长度 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details
 */
int32_t cm_uart_read(cm_uart_dev_e dev, void *data, int32_t len, int32_t timeout);

/**
 * @brief 获取串口配置
 *
 * @param [in] dev 串口设备ID
 * @param [out] cfg 串口配置
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details 用户自行维护cfg内存空间
 */
int32_t cm_uart_get_cfg(cm_uart_dev_e dev, cm_uart_cfg_t *cfg);

/**
 * @brief 获取串口缓存区待读数据长度
 *
 * @param [in] dev 串口设备ID
 *
 * @return
 * >= 0 - 待读数据长度 \n

```

```
* < 0 - 失败, 返回值为错误码
*
* @details
*/
int32_t cm_uart_get_rxrb_data_len(cm_uart_dev_e dev);

/**
* @brief 清空串口所有缓存区数据
*
* @param[in] dev 串口设备ID
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details 包含发送缓存和接收缓存
*/
int32_t cm_uart_clean(cm_uart_dev_e dev);
```



中国移动
China Mobile

5.3. 使用示例

UART DEMO提供了UART使用的功能示例程序，详细信息请参考SDK中的cm_demo_uart.c文件。

UART初始化

打开串口之前需要使用接口cm_iomux_set_pin_func()配置引脚复用功能，请参考使用示例以及cm_iomux.h文件中的注释说明。初始化串口示例如下：

```
int32_t ret = -1;

/* 配置参数 */
cm_uart_cfg_t config =
{
    CM_UART_BYTE_SIZE_8,
    CM_UART_PARITY_NONE,
    CM_UART_STOP_BIT_ONE,
    CM_UART_FLOW_CTRL_NONE,
    CM_UART_BAUDRATE_9600,
    0, //配置为普通串口模式，若要配置为低功耗模式可改为1
    0, //环形缓存区大小按照默认配置8k
    0,
    0,
};

/* 事件参数 */
cm_uart_event_t event =
{
    CM_UART_EVENT_TYPE_RX_ARRIVED|CM_UART_EVENT_TYPE_RX_OVERFLOW, //注册需要上报的事件类型
    "uart0", //用户参数
    cm_serial_uart_callback //上报事件的回调函数
};

/* 配置引脚复用 */
cm_iomux_set_pin_func(OPENCPU_TEST_UARTTX_IOMUX);
cm_iomux_set_pin_func(OPENCPU_TEST_UARTRX_IOMUX);

/* 注册事件和回调函数 */
ret = cm_uart_register_event(OPENCPU_MAIN_URAT, &event);

if (ret != RET_SUCCESS)
{
    cm_log_printf(0, "uart register event err, ret=%d\n", ret);
    return;
}

/* 开启串口 */
ret = cm_uart_open(OPENCPU_MAIN_URAT, &config);

if (ret != RET_SUCCESS)
{
    cm_log_printf(0, "uart init err, ret=%d\n", ret);
    return;
}
```

```

/* 配置串口唤醒 */
/* 只有UART0具有串口唤醒功能 */
if (OPENCPU_MAIN_URAT == CM_UART_DEV_0)
{
    /*
    配置uart唤醒功能，使能边沿检测才具备唤醒功能，仅主串口具有唤醒功能，用于唤醒的数据并不能被uart接收，请在唤醒后再进行
    uart数传 */
    cm_iomux_set_pin_cmd(OPENCPU_UART_WEKEUP_PIN, CM_IOMUX_PINCMD1_LPMEDEG,
    CM_IOMUX_PINCMD1_FUNC1_LPM_EDGE_RISE);
}

```

以上示例以及后续示例中的宏定义OPENCPU_TEST_UARTTX_IOMUX、OPENCPU_MAIN_URAT和OPENCPU_UART_WEKEUP_PIN请结合SDK中的代码查看。使用cm_iomux接口时需要查看资源综述确定使用第几号引脚的第几号功能，具体请查看对于文档和示例。

以上示例中使用cm_uart_register_event()注册了要接收的事件，未注册的事件将不会上报，也注册了上报用的回调函数cm_serial_uart_callback，回调函数示例如下：

```

/* 回调函数中不可输出LOG、串口打印、执行复杂任务或消耗过多资源，建议以信号量或消息队列形式控制其他线程执行任务 */
static void cm_serial_uart_callback(void *param, uint32_t type)
{
    uart_event_msg_t msg = {0};
    if (CM_UART_EVENT_TYPE_RX_ARRIVED & type)
    {
        /* 接收事件
        * TODO：触发其他线程执行读取数据，该部分请用户自行完成，可参考SDK中的代码。
        */
    }

    if ((CM_UART_EVENT_TYPE_RX_OVERFLOW & type))
    {
        /* 其他事件
        * TODO：触发其他线程处理其他事件，该部分请用户自行完成。
        */
    }
}

```

UART读取数据

串口接收到数据后，会将数据存入缓存区，缓存区大小默认为8192字节，用户可配置，存入缓存区后将通过回调函数上报接收事件后，此时应该使用cm_uart_read()从缓存区读取数据，若不读取，当缓存区存满后，将上报溢出事件，并丢弃数据，读取示例如下：

```

#define UART_BUF_LEN    1024
static char rx_rev_data[UART_BUF_LEN] = {0};

int read_len = 0;
read_len = cm_uart_read(OPENCPU_MAIN_URAT, (void*)rx_rev_data, UART_BUF_LEN, 0);

```

以上示例中read_len为读取接口的返回值，代表实际读取到的数据长度，当返回值为0时说明缓存区数据已被读完。

UART发送数据

使用cm_uart_write()发送数据，发送示例如下：

```
char *send_date = "uart send test\r\n";  
cm_uart_write(OPENCPU_MAIN_URAT, send_date, strlen(send_date), 0);
```

UART关闭

使用cm_uart_close()关闭串口，示例如下：

```
cm_uart_close(OPENCPU_MAIN_URAT);
```

5.4. 注意事项

使用UART应注意以下事项：

- 不同版本流控开放情况不同，请参考SDK中的头文件说明；
- 回调函数中不可输出LOG、串口打印、执行复杂任务或消耗过多资源，建议以信号量或消息队列形式控制其他线程执行任务；
- 针对普通模式串口，若有任何一路串口配置成特殊波特率，则分频时钟将会改变，即当前所有普通模式串口都只能在该特殊波特率下工作；
- 低功耗模式串口可在睡眠状态下正常收发，且使用单独的分频时钟故不受普通模式串口特殊波特率影响，但只支持115200及以下的标准波特率（不支持76800）；
- 只有UART0支持串口唤醒功能，串口唤醒时，用于唤醒的数据可能不会被串口正常接收。

6. SPI

本章主要介绍OpenCPU SDK中外设SPI的相关使用情况。

6.1. 硬件说明

OpenCPU SPI引脚映射详见对应版本的资源综述和硬件设计手册。

6.2. API说明

SDK中提供一套完整的SPI用户编程接口，具体接口定义和说明请查阅cm_spi.h头文件。

SPI参数结构体

```

/** 设备ID */
typedef enum
{
    CM_SPI_DEV_0, /*!< 设备1*/
    CM_SPI_DEV_1, /*!< 设备2*/
    CM_SPI_DEV_NUM
}cm_spi_dev_e;

/** SPI主从模式 */
typedef enum
{
    CM_SPI_MODE_SLAVE, /*!< SPI外设从机模式，暂不支持 */
    CM_SPI_MODE_MASTER /*!< SPI外设主机模式*/
}cm_spi_mode_e;

/** 模式 */
typedef enum
{
    CM_SPI_WOKR_MODE_0, /*!< CPOL = 0, CPHA = 0*/
    CM_SPI_WOKR_MODE_1, /*!< CPOL = 0, CPHA = 1*/
    CM_SPI_WOKR_MODE_2, /*!< CPOL = 1, CPHA = 0, 该模式下建议配置时钟低于 13MHZ或提高传输数据宽度，否则可能收发数据不连续*/
    CM_SPI_WOKR_MODE_3, /*!< CPOL = 1, CPHA = 1, 该模式下建议配置时钟低于 13MHZ或提高传输数据宽度，否则可能收发数据不连续*/
}cm_spi_work_mode_e;

/** SPI传输数据宽度 */
typedef enum
{
    CM_SPI_DATA_WIDTH_8BIT, /*!< SPI外设传输数据宽度8位 */
    CM_SPI_DATA_WIDTH_16BIT, /*!< SPI外设传输数据宽度16位，请注意字节对齐，传输字节数需为16bit倍数 */
    CM_SPI_DATA_WIDTH_32BIT, /*!< SPI外设传输数据宽度32位，请注意字节对齐，传输字节数需为32bit倍数*/
}cm_spi_data_width_e;

/** SPI的NSS管理模式 */
typedef enum
{
    CM_SPI_NSS_SOFT, /*!< SPI外设软件控制NSS，暂不支持*/
    CM_SPI_NSS_HARD, /*!< SPI外设硬件控制NSS*/
}cm_spi_nss_e;

/** 配置 */
typedef struct
{
    cm_spi_mode_e mode; /*!< SPI主从模式 */
    cm_spi_work_mode_e work_mode; /*!< 模式 */
    cm_spi_data_width_e data_width; /*!< SPI传输数据宽度 */
    cm_spi_nss_e nss; /*!< SPI的NSS管理模式 */
}

```

```
int clk;/** 时钟频率*/
} cm_spi_cfg_t;
```

SPI接口定义

```
/**
 * @brief 打开spi设备
 *
 * @param[in] dev SPI设备ID
 * @param[in] cfg 配置
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details 建议在mode2/3模式下配置clk低于13MHZ, 否则可能收发数据不连续。
 */
int32_t cm_spi_open(cm_spi_dev_e dev, cm_spi_cfg_t *cfg);

/**
 * @brief spi关闭, 去初始化
 *
 * @param[in] dev spi设备
 *
 * @return
 * = 0 - 成功.
 * < 0 - 失败, 返回值为错误码.
 *
 * @details More details
 */
int32_t cm_spi_close(cm_spi_dev_e dev);

/**
 * @brief SPI写数据
 *
 * @param[in] dev SPI设备ID
 * @param[in] data 待写数据
 * @param[in] len 待写数据长度
 *
 * @return
 * = 实际写入长度 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details 单次写数据不超过1024字节。
 */
int32_t cm_spi_write(cm_spi_dev_e dev, const void *data, int32_t len);

/**
 * @brief SPI读数据
 *
 * @param[in] dev SPI设备ID
 * @param[out] data 待读数据缓存
 * @param[in] len 长度
 *
 * @return
```

```

* = 实际读出长度 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details 单次读取数据不超过 1024 字节。
*/
int32_t cm_spi_read(cm_spi_dev_e dev, void *data, int32_t len);

/**
* @brief SPI 写数据后读出数据
*
* @param[in] dev SPI 设备 ID
* @param[in] w_data 待写数据
* @param[in] w_len 写入长度
* @param[out] r_data 待读缓存
* @param[in] r_len 读取长度
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details 单次读写长度之和不能超过 1024 字节。
*/
int32_t cm_spi_write_then_read(cm_spi_dev_e dev, const void *w_data, int32_t w_len, void *r_data, int32_t r_len);

```

6.3. 使用示例

SPI DEMO提供了SPI驱动外设的示例程序，详细信息请参考SDK中的cm_demo_spi.c文件。

SPI读写示例

```

/*****
 * Pre-processor Definitions
 *****/
#define CM_SPI_DEMO_LOG cm_demo_printf
#ifndef min
#define min(A,B) ((A) <= (B) ? (A) : (B))
#endif

#ifndef max
#define max(A,B) ((A) < (B) ? (B) : (A))
#endif

#define SPI_FLASH_BUS CM_SPI_DEV_1
#define SPI_PAGE_SIZE 256
#define SPI_SECTOR_SIZE 4096
#define SPI_RDID_CMD 0x9f
#define SPI_WREN_CMD 0x06
#define SPI_READ_CMD 0x03
#define SPI_PP_CMD 0x02
#define SPI_RDSR_CMD 0x05
#define SPI_SE_CMD 0x20

/*****
 * Private Types
 *****/

/*****
 * Private Function Prototypes
 *****/

/*****
 * Private Data
 *****/

/*****
 * Private Functions
 *****/

/*****
 * Public Functions
 *****/

static int spi_flash_read(uint32_t addr, char *data, int len);

static int spi_flash_read_status(uint8_t *status)
{
    char w_cmd[] = {SPI_RDSR_CMD};
    int ret = -1;

```

```

ret = cm_spi_write_then_read(SPI_FLASH_BUS, w_cmd, sizeof(w_cmd), (char *)status, 1);
osDelay(10);

return ret;
}

static bool spi_flash_is_busy(void)
{
    uint8_t l_status = {0};

    spi_flash_read_status(&l_status);
    osDelay(10);
    if (l_status & 0x01)
    {
        return true;
    }
    else
    {
        return false;
    }
}

static int spi_flash_read_id(uint32_t *id)
{
    char w_cmd[] = {SPI_RDID_CMD};
    char r_data[10] = {0};
    int ret = -1;

    ret = cm_spi_write_then_read(SPI_FLASH_BUS, w_cmd, sizeof(w_cmd), r_data, 3);
    *id = (r_data[0] << 16) + (r_data[1] << 8) + r_data[2];
    return ret;
}

static int spi_flash_write_enable(void)
{
    char w_cmd[] = {SPI_WREN_CMD};
    osDelay(10);

    return cm_spi_write(SPI_FLASH_BUS, w_cmd, 1);
}

static int spi_flash_page_write(uint32_t addr, char *data, int len)
{
    char w_cmd[300] = {SPI_PP_CMD, (char)(addr >> 16), (char)(addr >> 8), (char)addr};
    int i = 0;
    int ret = -1;

    for (i = 0; i < len; i++)
    {
        w_cmd[i + 4] = *data++;
    }

    ret = spi_flash_write_enable();//enable write

    if (ret < 0)

```

```

{
    return ret;
}
osDelay(10);
return cm_spi_write(SPI_FLASH_BUS, w_cmd, len + 4); //写入数据
}

static int spi_flash_sector_erase(uint32_t addr)
{
    char w_cmd[4] = {SPI_SE_CMD, (char)(addr >> 16), (char)(addr >> 8), (char)addr};

    if (spi_flash_write_enable() < 0)
    {
        return -1;
    }

    return cm_spi_write(SPI_FLASH_BUS, w_cmd, 4); //写入数据
}

static int spi_flash_write(uint32_t addr, char *data, int len)
{
    int ret = -1;
    char *tmp = cm_malloc(SPI_SECTOR_SIZE);
    int sector_start = addr / SPI_SECTOR_SIZE;
    int sector_end = (addr + len) / SPI_SECTOR_SIZE;
    int i = 0, j = 0, k = 0;

    if ((len <= 0) || (data == NULL))
    {
        cm_free(tmp);
        return -1;
    }

    for (i = sector_start; (i <= sector_end) && (len > 0); i++)
    {
        uint32_t current_sector_addr = i * SPI_SECTOR_SIZE;

        //写入数据至缓冲区
        int current_len = SPI_SECTOR_SIZE - (addr % SPI_SECTOR_SIZE);
        int count = min(len, current_len);
        int addr_start = addr % SPI_SECTOR_SIZE;
        int addr_end = addr_start + count;

        for (j = addr_start; j < addr_end; j++)
        {
            tmp[j] = *data++;
        }

        //擦除sector
        ret = spi_flash_sector_erase(current_sector_addr);

        if (ret < 0)
        {
            CM_SPI_DEMO_LOG("spi erase err\r\n");
            cm_free(tmp);
            return ret;
        }
    }
}

```

```

}

//cm_sleep_ms(150);
count = 0;
while (spi_flash_is_busy()) //等待擦除完成
{
    osDelay(10);
    count++;
    if(count > 200)
    {
        cm_free(tmp);
        return -1;
    }
}

//缓冲区写入,4kbyte/256 byte = 16 page
for (k = 0; k < 16; k++)
{
    ret = spi_flash_page_write(current_sector_addr + k * SPI_PAGE_SIZE, &tmp[k * SPI_PAGE_SIZE], SPI_PAGE_SIZE);
    if (ret < 0)
    {
        CM_SPI_DEMO_LOG("spi write err\r\n");
        cm_free(tmp);
        return ret;
    }
    count = 0;
    while (spi_flash_is_busy())
    {
        osDelay(10);
        count++;
        if(count > 200)
        {
            cm_free(tmp);
            return -1;
        }
    } //等待写入完成
}
//下一轮地址, 长度
addr = (i + 1) * SPI_SECTOR_SIZE;
len -= count;
}

cm_free(tmp);
return 0;
}

static int spi_flash_read(uint32_t addr, char *data, int len)
{
    char w_cmd[4] = {0};
    int ret = -1;

    while (len > 0) //每次最多读取0xffff字节数据
    {
        w_cmd[0] = SPI_READ_CMD;
        w_cmd[1] = (char)(addr >> 16);
        w_cmd[2] = (char)(addr >> 8);
    }

```

```

w_cmd[3] = (char)addr;

uint16_t count = min(0xffff, len);
ret = cm_spi_write_then_read(SPI_FLASH_BUS, w_cmd, sizeof(w_cmd), data, count);

if (ret != 0)
{
    CM_SPI_DEMO_LOG("spi read err");
    return ret;
}

addr += 0xffff;
len -= 0xffff;
data += 0xffff;
}

return 0;
}

/*****
* Public Functions
*****/
void cm_test_spi(unsigned char **cmd, int len)
{
    int ret = -1;
    cm_spi_cfg_t config = {CM_SPI_MODE_MASTER, CM_SPI_WOKR_MODE_0,
        CM_SPI_DATA_WIDTH_8BIT, CM_SPI_NSS_HARD, CM_SPI_CLK_3_25MHZ};
    uint32_t spi_id = 0;
    uint32_t test_addr = 0;
    char test_str[] = "Hello world!";

    int test_len = strlen(test_str);
    char tmp[1000] = {0};
    CM_SPI_DEMO_LOG("spi test running:%d\r\n", SPI_FLASH_BUS);

    ret = cm_spi_open(SPI_FLASH_BUS, &config);
    if (ret != 0)
    {
        CM_SPI_DEMO_LOG("spi init err:%d\r\n", ret);
        return -1;
    }

    //读取spi id
    ret = spi_flash_read_id(&spi_id);
    if (ret != 0 || spi_id == 0xffffffff)
    {
        CM_SPI_DEMO_LOG("spi read id ret:%d spi_id:0x%x err\r\n", ret, spi_id);
        goto spi_stop;
    }
    CM_SPI_DEMO_LOG("spi id = 0x%x\r\n", spi_id);

    //写入测试数据
    ret = spi_flash_write(test_addr, test_str, test_len);
    if (ret != 0)
    {
        CM_SPI_DEMO_LOG("spi write str err:%d\r\n", ret);
        goto spi_stop;
    }
}

```

```
}  
CM_SPI_DEMO_LOG("spi write len = %d\n", test_len);  
//读取数据  
ret = spi_flash_read(test_addr, tmp, test_len);  
  
if (ret != 0)  
{  
    CM_SPI_DEMO_LOG("spi read err:%d\n", ret);  
    goto spi_stop;  
}  
CM_SPI_DEMO_LOG("spi read len = %d,%s\n", strlen(tmp), tmp);  
spi_stop:  
cm_spi_close(SPI_FLASH_BUS);  
  
return 0;  
}
```



中国移动
China Mobile

6.4. 注意事项

使用SPI应注意以下事项：

- SPI仅支持主机模式。

7. I2C

本章主要介绍OpenCPU SDK中外设I2C的相关使用情况。

7.1. 硬件说明

OpenCPU I2C引脚映射详见对应版本的资源综述和硬件设计手册。

7.2. API说明

SDK中提供一套完整的I2C用户编程接口（不同SDK版本可能存在差异），具体接口定义和说明请查阅SDK中的cm_i2c.h头文件。

I2C参数结构

```
/**
 * @brief I2C 通道，一共3个通道
 */
typedef enum
{
    CM_I2C_DEV_0 = 0, /*!< I2C0 */
    CM_I2C_DEV_1, /*!< I2C1 */
    CM_I2C_DEV_2, /*!< I2C2 */
    CM_I2C_DEV_NUM, /*!< I2C设备个数 */
} cm_i2c_dev_e;

/**
 * @brief I2C 支持主从模式
 */
typedef enum
{
    CM_I2C_MODE_MASTER = 0, /*!< 主模式，仅支持 */
    CM_I2C_MODE_SLAVER, /*!< 从模式 */
} cm_i2c_mode_e;

/** 寻址 */
typedef enum
{
    CM_I2C_ADDR_TYPE_7BIT, /*!< 7bit地址，仅支持 */
    CM_I2C_ADDR_TYPE_10BIT, /*!< 10bit地址 */
} cm_i2c_addr_type_e;

/**
 * @brief I2C 配置
 */
typedef struct
{
    cm_i2c_addr_type_e addr_type;
    cm_i2c_mode_e mode;
    int clk;
} cm_i2c_cfg_t;
```

I2C接口定义

```
/**
 * @brief i2c 开启
 *
 * @param [in] dev i2c 设备ID
 * @param [in] cfg i2c 配置
 * @return
 * = 0 - 成功.
```

```

* < 0 - 失败, 返回值为错误码.
*
* @details 仅支持主机模式.
*/
int32_t cm_i2c_open(cm_i2c_dev_e dev, cm_i2c_cfg_t *cfg);

/**
* @brief i2c关闭
*
* @param[in] dev i2c设备ID
* @return
* = 0 - 成功.
* < 0 - 失败, 返回值为错误码.
*
* @details More details
*/

int32_t cm_i2c_close(cm_i2c_dev_e dev);

/**
* @brief i2c写数据
*
* @param[in] dev i2c设备
* @param[in] slave_addr i2c设备地址
* @param[in] data 待写数据
* @param[in] len 待写数据长度, 单次最大只支持写入8个字节
* @return
* = 实际写入长度 - 成功.
* < 0 - 失败, 返回值为错误码.
*
* @details More details
*/
int32_t cm_i2c_write(cm_i2c_dev_e dev, uint16_t slave_addr, const void *data, uint32_t len);

/**
* @brief i2c读数据
*
* @param[in] dev i2c设备
* @param[in] slave_addr i2c设备地址
* @param[out] data 读取数据存放地址, 需先申请内存
* @param[in] len 读取数据长度, 单次最大只支持读取8个字节
* @return
* = 实际读出长度 - 成功.
* < 0 - 失败, 返回值为错误码.
*
* @details More details
*/
int32_t cm_i2c_read(cm_i2c_dev_e dev, uint16_t slave_addr, void *data, uint32_t len);

```

7.3. 使用示例

I2C DEMO提供了I2C驱动外设的示例程序（不同SDK版本可能存在差异），详细信息请参考SDK中的cm_demo_i2c.c文件。

I2C读写示例

```

/*****
 * Pre-processor Definitions
 *****/
#define CM_DEMO_I2C_LOG cm_demo_printf

#define EPROM_DEV_ADDR 0x50
#define EPROM_I2C_ID CM_I2C_DEV_0

/*****
 * Private Types
 *****/

/*****
 * Private Function Prototypes
 *****/

/*****
 * Private Data
 *****/

/*****
 * Private Functions
 *****/
/**
 * @brief eeprom写1字节数据
 *
 * @param[in] addr:地址，范围0-0x7fff
 * @param[in] data:待写入数据
 *
 * @return
 * = 0 - 成功\n
 * < 0 - 失败, 返回值为错误码.
 *
 * @details More details
 */
static int32_t is24c256_write_byte(uint16_t addr, int8_t data)
{
    int8_t tmp[3] = {0};
    int32_t ret = -1;

    tmp[0] = (addr >> 8) & 0xff;
    tmp[1] = addr & 0xff;
    tmp[2] = data;

    ret = cm_i2c_write (EPROM_I2C_ID, EPROM_DEV_ADDR, tmp, 3);

    if (ret < 0)

```

```

{
    cm_demo_printf("i2c%d write e2prom err:%d\r\n",g_dev_i2c, ret);
    return RET_ERROR;
}

return RET_SUCCESS;
}

/**
 * @brief eeprom读1字节数据
 *
 * @param [in] addr:地址，范围0-0x7fff
 * @param [out] data:待读取数据指针
 *
 * @return
 * = 0 - 成功\n
 * < 0 - 失败, 返回值为错误码.
 *
 * @details More details
 */
static int32_t is24c256_read_byte(uint16_t addr, int8_t* data)
{
    int8_t tmp[2] = {0};
    int32_t ret;

    if(data == NULL)
    {
        cm_demo_printf("is24c256_read_bytedata ptr err\r\n");
        return RET_ERROR;
    }

    tmp[0] = (addr >> 8) & 0xff;
    tmp[1] = addr & 0xff;

    ret = cm_i2c_write(EEPROM_I2C_ID, EEPROM_DEV_ADDR, tmp, 2);
    if(ret < 0)
    {
        cm_demo_printf("i2c read addr err(w):%d\r\n", ret);
        return RET_ERROR;
    }

    ret = cm_i2c_read(EEPROM_I2C_ID, EEPROM_DEV_ADDR, data, 1);

    if(ret < 0)
    {
        cm_demo_printf("i2c read addr err(r):%d\r\n", ret);
        return RET_ERROR;
    }

    return RET_SUCCESS;
}

/*****
 * Public Functions
 *****/
void cm_test_i2c(unsigned char **cmd, int len)

```

```

{
    cm_i2c_cfg_t config =
    {
        CM_I2C_ADDR_TYPE_7BIT,
        CM_I2C_MODE_MASTER, //目前仅支持模式
        CM_I2C_CLK_100KHZ
    }; //master模式, (100KHZ)

    uint16_t eprom_addr = 0x100; //选取所支持的任意E2PROM地址进行测试, 可修改
    int8_t w_data = 'B'; r_data = 0;
    int32_t ret;

    CM_DEMO_I2C_LOG("i2c test start, i2c num:%d!\n", EPROM_I2C_ID);

    ret = cm_i2c_open(EPROM_I2C_ID, &config);
    if (ret != 0)
    {
        CM_DEMO_I2C_LOG("i2c init err, ret=%d\n", ret);
        return -1;
    }
    CM_DEMO_I2C_LOG("i2c init ok\n");

    //写入测试数据
    ret = is24c256_write_byte(eprom_addr, w_data);
    if (ret != 0)
    {
        CM_DEMO_I2C_LOG("i2c write e2prom err %d\n", (uint32_t)ret);
        cm_i2c_close(EPROM_I2C_ID);
        return -1;
    }
    CM_DEMO_I2C_LOG("i2c write e2prom ok\n");
    osDelay(10); //延时等待写入完成

    //读取数据
    ret = is24c256_read_byte(eprom_addr, &r_data);
    cm_i2c_close(EPROM_I2C_ID);

    if (ret != 0)
    {
        CM_DEMO_I2C_LOG("i2c read e2prom err: %d\n", ret);
        return -1;
    }

    CM_DEMO_I2C_LOG("i2c read e2prom, %c\n", r_data);
    CM_DEMO_I2C_LOG("i2c test end!\n");
    return 0;
}

```

7.4. 注意事项

使用I2C应注意以下事项：

- I2C模块只支持主模式。
- I2C模块只支持7bit地址。

8. PWM

本章主要介绍OpenCPU SDK中外设PWM的相关使用情况。

8.1. 硬件说明

OpenCPU PWM引脚映射详见对应版本的资源综述和硬件设计手册。



中国移动
China Mobile

8.2. API说明

SDK中提供一套完整的PWM用户编程接口，具体接口定义和说明请查阅cm_pwm.h头文件，并以SDK为准，这里不再赘述。

PWM参数结构体

```
/** 设备ID, 详情参照资源综述 */
typedef enum{
    CM_PWM_DEV_0, /*!< 设备0*/
    ... ..
    CM_PWM_DEV_NUM
} cm_pwm_dev_e;

/** 时钟源枚举 */
typedef enum{
    CM_PWM_CLK_32K = 32000,    /*!< 32k时钟源, 支持低功耗下使用*/
    CM_PWM_CLK_12800K = 12800000, /*!< 12.8M时钟源, 不支持低功耗下使用*/
    CM_PWM_CLK_END
} cm_pwm_clk_e;
```

PWM接口定义

```
/**
 * @brief PWM 时钟源选择
 *
 * @param [in] dev PWM设备ID
 * @param [in] clk 时钟源
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details clk可以选择32k的时钟, 也可以选择12.8M的时钟, 低功耗下仅支持32k的时钟; 需要在open函数之前设置
 */
int32_t cm_pwm_set_clk(cm_pwm_dev_e dev, cm_pwm_clk_e clk);

/**
 * @brief 打开PWM设备
 *
 * @param [in] dev PWM设备ID
 * @param [in] period 周期(ns)
 * @param [in] period_h 周期高电平占用时间(ns)
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败, 返回值为错误码
 *
 * @details PWM由于分频和算法设计存在一定的误差, 受限与软件和硬件, 因此period和period_h需根据下面备注进行设置\n
 * How to get the PV(The value of after scaled clock cycles per cycle/T):\n
 * InputClockT = 13M; PWM_T = period;\n
 * InputClockT * (PV(The value of before scaled clock cycles per cycle)) = PWM_T\n
 * 'PV = PWM_T / InputClockT\n
 * = PWM_T * InputClockF\n
 */
```

```

* PV = 'PV / (prescale + 1) - 1 \n
* How to get the prescale: \n
* Because the internal clock period counter is 10-bit, to avoid overrun. We can use the prescale.\n
* prescale real value = ('PV - 1 (if > 1024, then need prescale)) / 1024 \n
*
* How to get the Duty cycle: \n
* Duty cycle = (PV(the cycles per T) + 1) * ratio(Duty time/PWM_T)
*/
int32_t cm_pwm_open_ns(cm_pwm_dev_e dev, uint32_t period, uint32_t period_h);

/**
* @brief 关闭PWM设备
*
* @param [in] dev PWM设备ID
*
* @return
* = 0 - 成功 \n
* < 0 - 失败, 返回值为错误码
*
* @details More details
*/
int32_t cm_pwm_close(cm_pwm_dev_e dev);

```



8.3. 使用示例

PWM DEMO提供了PWM使用的功能示例程序，详细信息请参考SDK中的cm_demo_pwm.c文件，并以SDK为准，这里不再赘述。

PWM示例程序

PWM功能使用步骤：

1. 将相应的PIN脚设置为PWM的功能；
2. 设置PWM的时钟源，仅32K时钟源支持低功耗下输出；
3. 关闭PWM的功能。

8.4. 注意事项

使用PWM应注意以下事项：

- PWM因为分频的原因，存在一定的误差，所以需要根据备注公式自行计算最优输入参数；
- PWM的主频为12.8M/32K，可以通过函数对其进行设置；
- PWM 32K时钟支持低功耗模式下输出，12.8M时钟不支持低功耗模式下输出；
- 使用时需根据时钟频率调整输入参数的值，才能得到更准确的PWM波形。

9. KEYPAD

本章主要介绍OpenCPU SDK中外设KEYPAD的相关使用情况。

9.1. 硬件说明

OpenCPU KEYPAD引脚映射详见对应版本的资源综述和硬件设计手册。



中国移动
China Mobile

9.2. API说明

SDK中提供一套完整的KEYPAD用户编程接口，具体接口定义和说明请查阅cm_keypad.h头文件，并以SDK为准，这里不再赘述。

KEYPAD接口定义

```
/**
 * @brief key map
 *
 */
typedef uint32_t cm_keypad_map_t;

/**
 * @brief key 事件类型
 *
 */
typedef enum
{
    CM_KEY_EVENT_RELEASE = 0, /*!<按键被释放*/
    CM_KEY_EVENT_PRESS = 1, /*!<按键被按下*/
} cm_keypad_event_e;

/*具体配置参照资源综述*/
typedef enum cm_keypad_kp_mkin{
    CM_KP_MKI0 = 0,
    CM_KP_MKI1,
    CM_KP_MKI2,
    CM_KP_MKI3,
    CM_KP_MKI4,
    CM_KP_MKI5,
    CM_KP_MKI6,
    CM_KP_MKI7,
    CM_KP_MKI_NO_VALID
}cm_keypad_kp_mkin_e;

/*具体配置参照资源综述*/
typedef enum cm_keypad_kp_mkout{
    CM_KP_MKO0 = 0,
    CM_KP_MKO1,
    CM_KP_MKO2,
    CM_KP_MKO3,
    CM_KP_MKO4,
    CM_KP_MKO5,
    CM_KP_MKO6,
    CM_KP_MKO7,
    CM_KP_MKO_NO_VALID
}cm_keypad_kp_mkout_e;

/** 配置 */
typedef struct{
    cm_keypad_kp_mkin_e *cm_col;
```

```

cm_keypad_kp_mkout_e *cm_row;
uint8_t cm_col_len; /*cm_col长度
uint8_t cm_row_len; /*cm_cow长度
} cm_keypad_cfg_t;

/**
 * @brief keypad事件回调函数
 *
 * @param[in] key 键值
 * @param[in] event 按键事件
 *
 */
typedef void (*cm_key_event_callback_t)(cm_keypad_map_t key, cm_keypad_event_e event);

/**
 * @brief keypad配置
 *
 * @return
 * = 0 - 成功 \n
 * = -1 - 失败
 *
 * @details 需在cm_keypad_init函数之前使用
 */
int32_t cm_keypad_config(cm_keypad_cfg_t *cm_keypad_cfg);

/**
 * @brief keypad初始化
 *
 * @return
 * = 0 - 成功 \n
 * = -1 - 失败
 */
int32_t cm_keypad_init(void);

/**
 * @brief keypad事件回调注册
 *
 * @param[in] cb keypad回调函数
 *
 * @return
 * = 0 - 成功 \n
 * = -1 - 失败
 *
 * @details 最大允许注册的回调函数个数为10
 */
int32_t cm_keypad_register(cm_key_event_callback_t cb);

/**
 * @brief keypad事件回调注销
 *
 * @param[in] cb keypad回调函数
 *
 * @return

```

```
* = 0 - 成功 \n
* = -1 - 失败
*
* @details 最大允许注册的回调函数个数为 10
*/
int32_t cm_keypad_unregister(cm_key_event_callback_t cb);

/**
* @brief keypad去初始化
*
*/
void cm_keypad_deinit(void);
```



中国移动
China Mobile

9.3. 使用示例

KEYPAD DEMO提供了KEYPAD的示例程序，详细信息请参考SDK中的cm_demo_keypad.c文件，并以SDK为准，这里不再赘述。

KEYPAD按键示例

功能使用步骤：

1. 将KEYPAD相应的PIN脚设置为KEYPAD的功能；
2. 设置KEYPAD引脚的SLEEP寄存器为SLEEP_DIR；
3. 注册KEYPAD回调函数；
4. 配置KEYPAD的参数；
5. 配置低功耗唤醒和休眠回调函数回调函数（因为KEYPAD在低功耗和唤醒状态使用的是不同的中断函数，因此需要在低功耗唤醒和休眠回调函数回调函数中使用cm_keypad_sleep()进行切换）；
6. 初始化KEYPAD功能。

9.4. 注意事项

使用KEYPAD应注意以下事项：

- cm_keypad_register()接口最大允许注册的回调函数个数为10；
- 每个按键对应的键值（即cm_keypad_map_t key）与实际硬件电路接线相关；
- 检测到KEYPAD按键操作时，底层采用中断回调的机制将KEYPAD状态回调至上层应用，KEYPAD回调函数不可阻塞；
- 按键数量可配，参照资源综述引脚定义；
- 设备上电之后首先需要将各个引脚功能进行IOMUX配置，特别时keypad引脚；
- 不使用的keypad引脚需要IOMUX配置为其他功能，多个引脚不能同时配置为同一功能；
- 中断函数不能处理耗时和重进入函数，例如打印；
- 如果需要keypad唤醒功能，需要在进入低功耗回调函数中设置keypad休眠使能；
- 不能将无关引脚接入矩阵键盘，特别是将可以配置为键盘的引脚接入，这将引入不确定电平干扰寄存器判断；
- 按键值需要用16进制表示，高4位为行号，后4位为列号。

10. 编程设计注意

外围接口一般与硬件强相关，需要模组匹配合理的外设设备，且进行软件开发前需详细阅读关联的硬件设计手册。

编程设计时应注意以下几点：

- 对存在引脚复用的外设，应先确认引脚功能以及定义复用功能；
- 单个引脚仅复用其中一种功能，详情参考《资源综述》。